Event Sourcing & CQRS

Bank account ledger + balance read model
Season 4 · Enterprise · Advanced

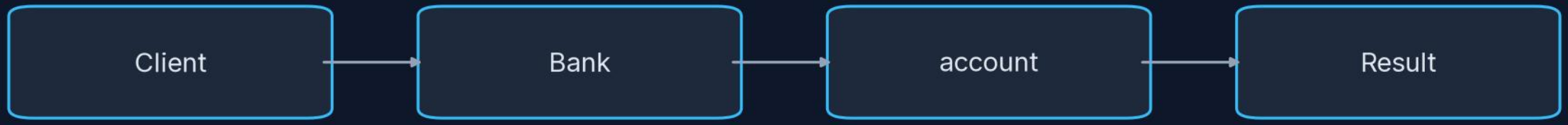
Today's Hook

- Problem: Bank account ledger + balance read model
- Naive pain cue: caught=funds
- We will refactor to Event Sourcing/CQRS with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Bank account ledger + balance read model
- Constraints: solve «Bank account ledger + balance read model» with Event Sourcing/CQRS — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Bank account ledger + balance read model
- Success looks like: bal=60

Structure



Event Sourcing/CQRS: Bank account ledger + balance read model

Real-World Scenario

- Running example: Bank account ledger + balance read model
- Key types: Event, Deposited, Withdrawn, Account
- Happy-path stdout cue: bal=60
- Failure-path stdout cue: caught=funds

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;

import java.util.*;

public final class NaiveAccount {
  private int bal; void deposit(int c){ bal+=c; } void withdraw(int c){ bal-=c; } int bal(){ return bal; }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Bank account ledger + balance read model
- Without Event Sourcing/CQRS, call sites absorb the wrong details.
- Failure mode to show on camera: caught=funds
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Event permits Deposited, Withdrawn {}
record Deposited(int cents) implements Event {}
record Withdrawn(int cents) implements Event {}
final class Account {
  private final List<Event> events=new ArrayList<>();
  void deposit(int c){ if(c<=0) throw new IllegalArgumentException("deposit"); events.add(new Deposited(c)); }
  void withdraw(int c){ if(c<=0 || balance()<c) throw new IllegalStateException("funds"); events.add(new Withdrawn(c)); }
  int balance(){ int b=0; for(Event e: events){ if(e instanceof Deposited d) b+=d.cents(); else if(e instanceof Withdrawn w) b-=w.cents(); } return b; }
  List<Event> history(){ return List.copyOf(events); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Event Sourcing: bank ledger + balance ===");
    System.out.println("--- naive ---"); var n=new NaiveAccount(); n.deposit(100); n.withdraw(40); System.out.println(n.bal());
System.out.println("--- event sourcing ---");
var a=new Account(); a.deposit(100); a.withdraw(40); System.out.println("bal="+a.balance()); System.out.println("events="+a.history());
try { a.withdraw(1000); } catch(IllegalStateException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Event Sourcing: bank ledger + balance ===
--- naive ---
60
--- event sourcing ---
bal=60
events=[Deposited[cents=100], Withdrawn[cents=40]]
caught=funds
```

Step-by-Step Trace

- Match stdout to stages — look for: bal=60
- Then the injected failure: caught=funds
- Re-run: java -cp ... 32-event-sourcing-cqrs ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Event permits Deposited, Withdrawn {}
record Deposited(int cents) implements Event {}
record Withdrawn(int cents) implements Event {}
final class Account {
  private final List<Event> events=new ArrayList<>();
  void deposit(int c){ if(c<=0) throw new IllegalArgumentException("deposit"); events.add(new Deposited(c)); }
  void withdraw(int c){ if(c<=0 || balance()<c) throw new IllegalStateException("funds"); events.add(new Withdrawn(c)); }
  int balance(){ int b=0; for(Event e: events){ if(e instanceof Deposited d) b+=d.cents(); else if(e instanceof Withdrawn w) b-=w.cents(); } return b; }
  List<Event> history(){ return List.copyOf(events); }
}
```

Trade-offs

- Use Event Sourcing/CQRS when «Bank account ledger + balance read model» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Event Sourcing/CQRS in product language.

Related Patterns

- Name the pattern usually confused with Event Sourcing/CQRS and separate the forces.
- Sketch the roles for «Bank account ledger + balance read model» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Bank account ledger + balance read model
- Naive pain → Event Sourcing/CQRS boundary → Demo stdout.
- Remember the failure cue: caught=funds
- Break a ProblemTest assertion, watch it fail, restore it.